

A real-time video agent that plays tic-tac-toe with a human

Role: VP, Research & Development

Time-box: Around 4 hours, your stack. A finished loop with honest notes beats an ambitious system that never plays.

1. Instructions

Setup. A webcam or phone camera is pointed at a paper 3×3 grid. The agent observes the page continuously, in real time, at low latency.

The task. Build a system in which a human plays tic-tac-toe against an AI agent, using the paper as the board. The human marks a cell by hand. The agent detects the new mark without being prompted, resolves the current board state, selects a legal reply, and communicates which cell it plays so the human can draw it. The loop repeats until the game reaches a win or a draw, with the final state on the page matching the state the agent reports.

The human is the only hand. The agent owns perception, decision, and session state. Ink on the page is the move: a mark the agent has not seen drawn does not exist.

Your decisions. How you locate and rectify the grid, how you decide a new mark has appeared, what you do when the page is ambiguous and your confidence is low, where any expensive inference sits in the loop, your latency and cost targets, and the output modality (spoken, displayed, or both) are all yours. Document each choice and the reasoning behind it. We would rather see a defended pick than a fashionable default.

You should not need to train or fine-tune a model to do this well. If you choose to, tell us why it earned its place.

Where something is ambiguous, that is partly intentional. State your assumptions and proceed.

2. Architecture design

One page, no more. A visual diagram (Mermaid, hand-drawn, or any tool you prefer) expanding what you built into a game-agnostic system: one that accepts the rules of an arbitrary game and plays it against a human over live video. A diagram plus a few lines per point is enough, including for the parts you did not build.

- **End-to-end flow.** The path from a frame arriving to a move being communicated, where inference runs, and which models or vendors you would use.
- **Services and boundaries.** How perception and decision run as separate services, which of them actually needs a model and which does not, what crosses between them, and what state each owns.
- **Shared and per-tenant.** What is common across all games and all users, and what is scoped to one: weights, rules, learned state, endpoints.
- **Failure.** What breaks first, and how you would know.

Deliverables

A shareable repo or a zip containing:

1. Video of a full game played end to end, with the human and the page in frame.
2. Source code.
3. A README that takes a stranger to a running game in under ten minutes.
4. The one-page architecture design.

Working with AI tools

Using an AI assistant or coding agent (Claude, Gemini, Cursor, and so on) is encouraged. We assume you will, and part of what we look at is how well you direct it and how sharply you edit the result. The bar is that you can open any file we point at and explain why it is there, what it does not do, and what you would change.

If you have energy left (genuinely optional). Make the agent get better across games, and tell us how you would know it actually did rather than got lucky.